```
  desc "This generator creates an initializer file at
config/initializers"
  def create_initializer_file
    create_file "config/initializers/initializer.rb", "# Add
initialization content here"
  end
end
```

Now we can see the new description by invoking `--help` on the new generator. The second way to add a description is by creating a file named `USAGE` in the same directory as our generator. We are going to do that in the next step.

3 Creating Generators with Generators

Generators themselves have a generator:

```
$ bin/rails generate generator initializer
      create  lib/generators/initializer
      create  lib/generators/initializer/initializer_generator.rb
      create  lib/generators/initializer/USAGE
      create  lib/generators/initializer/templates
```

This is the generator just created:

```
class InitializerGenerator < Rails::Generators::NamedBase
  source_root File.expand_path("../templates", __FILE__)
end
```

First, notice that we are inheriting from `Rails::Generators::NamedBase` instead of `Rails::Generators::Base`. This means that our generator expects at least one argument, which will be the name of the initializer, and will be available in our code in the variable `name`.

We can see that by invoking the description of this new generator (don't forget to delete the old generator file):

```
$ bin/rails generate initializer --help
Usage:
  rails generate initializer NAME [options]
```

We can also see that our new generator has a class method called `source_root`. This method points to where our generator templates will be placed, if any, and by default it points to the created directory `lib/generators/initializer/templates`.

In order to understand what a generator template means, let's create the file `lib/generators/initializer/templates/initializer.rb` with the following content:

```
# Add initialization content here
```

And now let's change the generator to copy this template when invoked:

```
class InitializerGenerator < Rails::Generators::NamedBase
  source_root File.expand_path("../templates", __FILE__)

  def copy_initializer_file
    copy_file "initializer.rb", "config/initializers/#{file_name}.rb"
  end
end
```

And let's execute our generator:

```
$ bin/rails generate initializer core_extensions
```

We can see that now an initializer named core_extensions was created at `config/initializers/core_extensions.rb` with the contents of our template. That means